

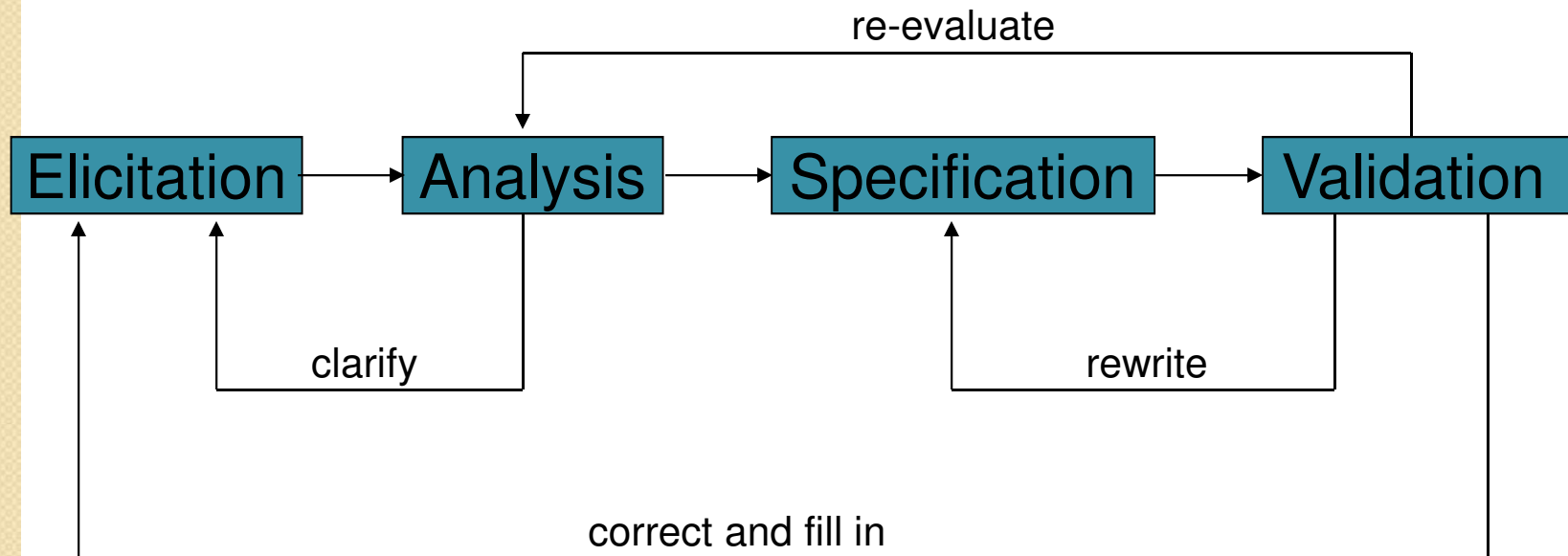


Chapter 3:

Good Practices for Requirements Engineering

A Process

- Be iterative, incremental, interleaved (overlapped)





3.1 Elicitation, I

- Define the requirements development process—its steps and activities.
- Write a vision and scope document
 - define objectives
 - set boundaries
 - focus the project



Elicitation, 2

- Identify user classes
 - Roles
 - Goals
 - Features used
 - Frequency of use
 - Knowledge and skill levels
 - Location, attitude



Elicitation, 3

- Product champions from user classes
 - Representatives of group
 - Speak for them
 - Decide on their behalf
 - Long-term commitment to project
- Focus groups from user classes
 - Describe functionality and quality needs
 - Short-term commitment



Elicitation, 4

- Identify use cases
 - Tasks to be accomplished
 - Goals and purpose
 - User/system interactions
- Identify system events/responses
 - External signals and data
 - Temporal events
 - Business events (customer calls, inventory changes,...)



Elicitation, 5

- Workshops
 - Collaboration between analyst and user
 - Discussions and draft documents
- Observe users
 - Current tasks establish context for new system
 - Data flow can be captured



Elicitation, 6

- Examine problem reports
 - Difficulties with old system can reveal needs for new one
 - Enhancements may have been explicitly requested
- Reuse requirements
 - Look for similarities to previous projects, existing products



3.2 Analysis, I

- Draw a context diagram
 - Show environment, boundaries, interfaces
- Create prototypes
 - User interfaces—to get feedback about a tangible entity
 - Technical—to explore feasibility of potential problem areas



3.2 Analysis, 2

- **Prioritize**
 - Allocate requirements to releases
 - Adjust to changes in resources, needs, goals, market conditions
- **Model**
 - Use an abstract, flexible representation
 - Use rigorous notation to reveal problems (incomplete, inconsistent, conflicting)



Analysis, 4

- Identify the interfaces btw. To other systems
- Identify the requirements to other subsystems.

3.3 Specification, I

- Documentation—consistent, accessible, reviewable
- Adopt SRS template—such as IEEE 830
- Identify sources
 - Justify presence of requirements
 - Support future clarification
- Label requirements
 - Have unique ID for traceability and management



Specification, 2

- Record business rules
 - Keep separate from SRS, they exist outside the scope of a single project.
- Specify quality attributes
 - Performance, efficiency, reliability, robustness, usability—all affect customer/user satisfaction.



3.4 Validation

- Inspect documents
 - Formal examinations by people with different perspectives and expertise.
 - Informal reviews of drafts in progress.
- Define test cases
 - Describe expected behavior
 - Review with customer/user
 - Trace to specification
 - Define acceptance criteria



3.5 Management, I

- Define change control process—proposal, analysis, resolution
- Establish a Change Control Board—small, competent, empowered group
- Perform impact analysis
 - scope of change (other artifacts affected)
 - effort and cost



Management, 2

- Establish baseline and version control
 - Distinguish between release baselines
 - Distinguish between previous and current versions
- Maintain a change history—know the what, when, who, and why
- Track change status—know every requirement's condition (proposed, approved, implemented, verified)



Management, 3

- Measure volatility
 - Know the rate of change
 - Identify problems (poor understanding, ill-defined scope, business dynamics, politics)
- Use a tool—automation eliminates drudgery, enables previously-described tasks



Management, 4

- Create a traceability matrix
 - Connect requirements, code, tests
 - Ensure no requirements are missed
 - Prevent extraneous features from appearing

3.6 Summaries

TABLE 3-1 Requirements engineering good practices

Elicitation	Analysis	Specification	Validation
■ Define vision and scope ■ Identify user classes ■ Select product champions ■ Conduct focus groups ■ Identify user requirements ■ Identify system events and responses ■ Hold elicitation interviews ■ Hold facilitated elicitation workshops ■ Observe users performing their jobs ■ Distribute questionnaires ■ Perform document analysis ■ Examine problem reports ■ Reuse existing requirements	■ Model the application environment ■ Create prototypes ■ Analyze feasibility ■ Prioritize requirements ■ Create a data dictionary ■ Model the requirements ■ Analyze interfaces ■ Allocate requirements to subsystems	■ Adopt requirement document templates ■ Identify requirement origins ■ Uniquely label each requirement ■ Record business rules ■ Specify nonfunctional requirements	■ Review the requirements ■ Test the requirements ■ Define acceptance criteria ■ Simulate the requirements
Requirements management	Knowledge	Project management	
■ Establish a change control process ■ Perform change impact analysis ■ Establish baselines and control versions of requirements sets ■ Maintain change history ■ Track requirements status ■ Track requirements issues ■ Maintain a requirements traceability matrix ■ Use a requirements management tool	■ Train business analysts ■ Educate stakeholders about requirements ■ Educate developers about application domain ■ Define a requirements engineering process ■ Create a glossary	■ Select an appropriate life cycle ■ Plan requirements approach ■ Estimate requirements effort ■ Base plans on requirements ■ Identify requirements decision makers ■ Renegotiate commitments ■ Manage requirements risks ■ Track requirements effort ■ Review past lessons learned	

Development Process in Summary:

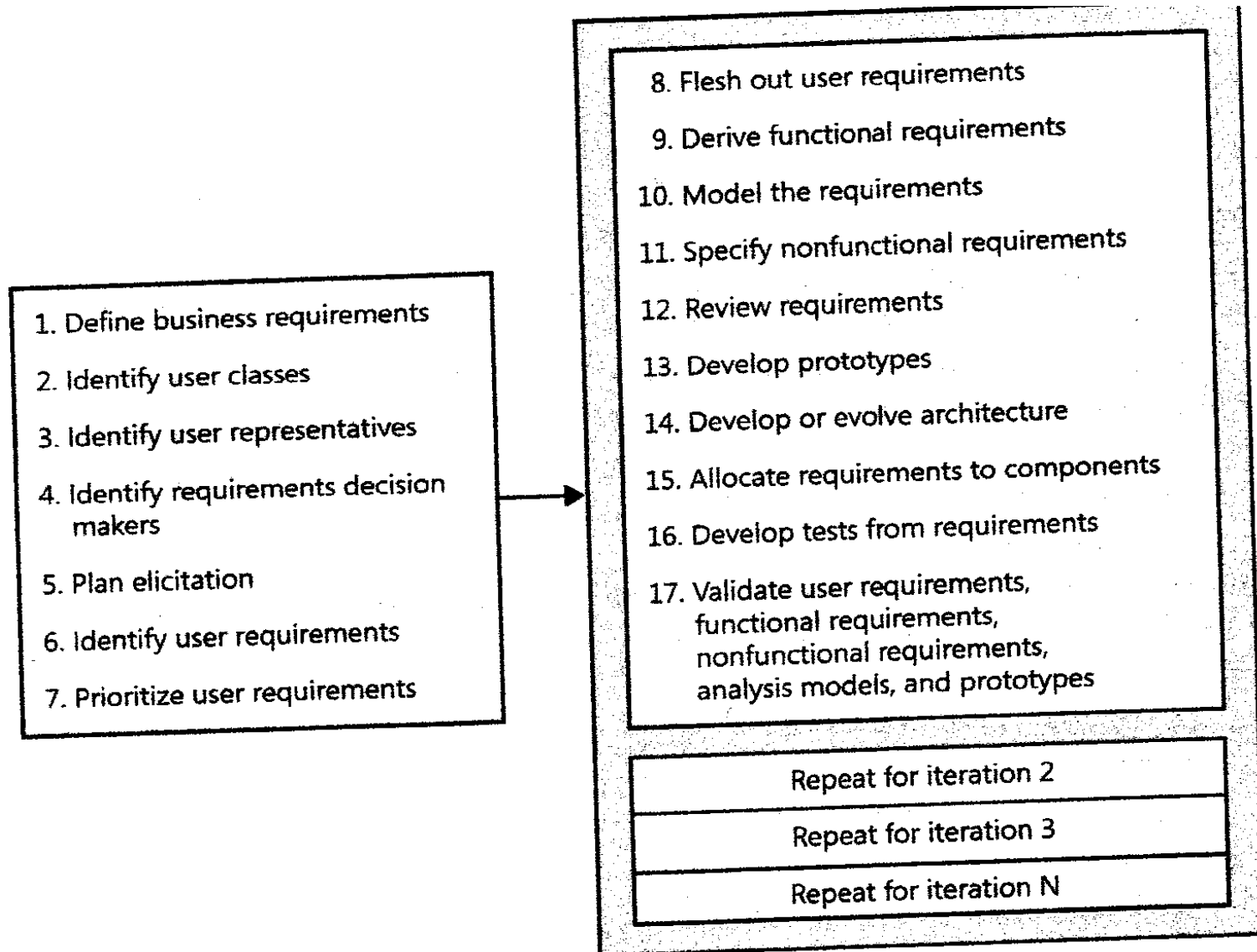


FIGURE 3-2 A representative requirements development process.